浙江大学实验报告

专业：电子科学与技术
姓名：郑智瑄
学号：3230103913
日期：March 6, 2026
地点：东 4-216

课程名称：人工智能实验　　指导老师：魏准　　实验类型：设计型实验
实验名称：实验 1-1 分类模型　　成　绩：　　签　名：

装 订 线

一、 实验目的

(1) 完成深度学习基础开发环境的搭建，包括 VS Code、Anaconda 及 TensorFlow。

(2) 理解 k-近邻（k-NN）算法的工作原理，掌握其基本编程实现。

(3) 通过 AB 分类实验验证算法在简单数据集上的分类效果。

二、 实验环境配置

本阶段主要完成开发工具的安装与虚拟环境的隔离，参考了教程：https://blog.csdn.net/2301_80035882/article/details/158389462。

(1) **编辑器**：安装 VS Code，并配置 Python。

(2) **包管理**：安装 Anaconda，通过 `conda` 创建独立的虚拟环境，避免版本冲突。

(3) **深度学习框架**：在虚拟环境中安装 TensorFlow 2.x 版本。经查阅匹配关系，本实验采用 Python 3.9.12 与 TensorFlow 2.10.0 的组合。TensorFlow 2.10 是最后一个支持原生 Windows 上 GPU 的版本，因此选择该版本以获得最佳兼容性。

(4) **GPU 配置**：针对 NVIDIA 显卡安装 CUDA 11.2 与 cuDNN 8.1，并将相关路径添加至系统环境变量中。

三、 k-近邻算法原理

k-近邻算法通过测量测试样本与训练样本集之间的特征距离进行分类。其核心步骤如下：

(1) 计算输入向量与训练集中各向量的欧氏距离：$d = \sqrt{\sum(a_i - b_i)^2}$。

(2) 按照距离由小到大排序，提取前 k 个最近邻样本。

(3) 统计这 k 个样本中各类别出现的频率，将出现次数最多的类别作为最终分类结果。

四、 实验 1-1：AB 分类实现

1. 算法核心代码 (kNN.py)

实现基础的数据集创建函数 `createDataSet` 和分类核心函数 `classify0`。

```python
from numpy import *
import operator

def createDataSet():
    # 创建训练集特征与标签
    group = array([[1.0, 1.1], [1.0, 1.0], [0, 0], [0, 0.1]])
    labels = ['A', 'A', 'B', 'B']
    return group, labels

def classify0(inX, dataSet, labels, k):
    dataSetSize = dataSet.shape[0]
    # 计算欧氏距离
    diffMat = tile(inX, (dataSetSize, 1)) - dataSet
    sqDiffMat = diffMat**2
    sqDistances = sqDiffMat.sum(axis=1)
    distances = sqDistances**0.5

    # 排序并取前 k 个点的标签频率
    sortedDistIndicies = distances.argsort()
    classCount = {}
    for i in range(k):
        voteIlabel = labels[sortedDistIndicies[i]]
        classCount[voteIlabel] = classCount.get(voteIlabel, 0) + 1

    # 返回频率最高类别
    sortedClassCount = sorted(classCount.items(),
                              key=operator.itemgetter(1), reverse=True)
    return sortedClassCount[0][0]
```

2. 测试模块 (tests.py)

编写测试脚本，验证不同 k 值下特定坐标点的类别归属。

```python
import kNN

group, labels = kNN.createDataSet()

# 测试点 [0, 0] 和 [0.8, 0.7]
test_points = [[0, 0], [0.8, 0.7]]
k_values = [1, 2, 3]

for pt in test_points:
    for kv in k_values:
        result = kNN.classify0(pt, group, labels, kv)
        print(f"输入点: {pt}, k: {kv}, 分类结果: {result}")
```

3. 运行输出

```
输入点: [0, 0], k: 1, 分类结果: B
输入点: [0, 0], k: 2, 分类结果: B
输入点: [0, 0], k: 3, 分类结果: B
输入点: [0.8, 0.7], k: 1, 分类结果: A
输入点: [0.8, 0.7], k: 2, 分类结果: A
输入点: [0.8, 0.7], k: 3, 分类结果: A
```

五、 实验结果与分析

(1) **环境验证**：成功在 VS Code Terminal 中激活 `tensorflow` 环境，并能正常导入 `numpy` 库。

(2) **分类结果**：

- 对于点 `[0, 0]`，由于其与训练集中 `B` 类样本重合，各 k 值下结果均为 `B`。
- 对于点 `[0.8, 0.7]`，该点位置更靠近 `A` 类区域，经计算返回结果为 `A`。

(3) **结论**：实验证明了 k-NN 算法在处理简单线性可分或聚类明显的特征数据时具有直观且有效的分类能力。

六、 遇到的问题与解决

1. NumPy 版本不兼容

问题描述：执行 `import tensorflow as tf` 时报错：

```
A module that was compiled using NumPy 1.x cannot be run in
NumPy 2.0.2 as it may crash. To support both 1.x and 2.x
versions of NumPy, modules must be compiled with NumPy 2.0.
```

原因分析：TensorFlow 2.10.0 依赖 NumPy 1.x 编译的底层模块，而环境中默认安装了 NumPy 2.0.2，导致二进制兼容性冲突。

解决方式：强制安装 NumPy 1.x 的最后一个稳定版本：

```
pip install "numpy<2" -i https://pypi.tuna.tsinghua.edu.cn/simple
```

安装完成后重新执行 `import tensorflow as tf`，问题解决。